Group Consecutive Numbers

Company: Google
Round: Ofline
Year: 2026
Difficulty: Medium
Topic: Arrays, HashMap, Greedy
Source: https://www.desiqna.in/19233/amazon-sde-2-recent-interview-experiences-2026-set-66

Problem Description

Given an array of numbers, check whether the numbers can be divided into groups of **5 consecutive increasing numbers**.
Every number must be used exactly once.
The output should be:
true
if the array can be divided correctly, otherwise:
false

Input Format

- An integer array nums.
- The array should be divided into groups of 5 consecutive numbers.
- Every element must be used exactly once.

Output Format

Return a boolean value:

- true if the array can be divided into valid groups.
- false otherwise.

Sample Input

nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

Sample Output

true

Explanation

The array can be divided as:

[1, 2, 3, 4, 5]

[6, 7, 8, 9, 10]

Both groups contain 5 consecutive numbers.

Approach to Solve This Problem:

- Count the frequency of every number using a HashMap.
- Find the smallest number that is still unused.
- Try to create a group of 5 consecutive numbers starting from that number.
- Decrease the frequency of each number used.
- If any required number is missing, return false.
- If all numbers are successfully used, return true.

Java Solution:

```java
import java.util.*;
public class Main {
    public static boolean canDivide(int[] nums) {
        if (nums.length % 5 != 0) {
            return false;
        }
        TreeMap<Integer, Integer> map = new TreeMap<>();
        for (int num : nums) {
            map.put(num, map.getOrDefault(num, 0) + 1);
        }
        while (!map.isEmpty()) {
            int first = map.firstKey();
            for (int i = 0; i < 5; i++) {
                int num = first + i;
                if (!map.containsKey(num)) {
                    return false;
                }
                map.put(num, map.get(num) - 1);
                if (map.get(num) == 0) {
                    map.remove(num);
                }
            }
        }
        return true;
    }
    public static void main(String[] args) {
        int[] nums = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
        System.out.println(canDivide(nums));
    }
}
```

Solution:

The main idea is to **sort the numbers and always start a new group with the smallest unused number**.

- First, check if the total number of elements is divisible by 5.
- Count how many times each number appears.
- Take the smallest unused number as the start of a group.
- Check whether the next **4 consecutive numbers** are available.
- If any number is missing, return false.
- Reduce the frequency of the five used numbers.
- Continue until all numbers are used.
- If all groups are formed successfully, return true

Complexity Analysis:

The array can be divided as:

Time Complexity: O(n log n)

We use a TreeMap, which keeps the numbers sorted. Inserting, searching, updating, and removing elements takes O(log n) time, repeated for n elements.

Space Complexity: O(n)

The TreeMap stores the frequency of the numbers.